

C++ Programming

Day 1: C++ 기초 문법

프로그램 구조, 입출력, 변수, 연산자, 조건문, 반복문

Contents

1	학습 목표	2
2	C++ 프로그램의 기본 구조	2
2.1	#include <iostream>	2
2.2	using namespace std;	2
2.3	int main()	2
2.4	return 0;	2
3	출력: cout	3
3.1	endl과 줄바꿈	3
3.2	예제: 학생 정보 출력	3
4	변수와 자료형	3
4.1	기본 자료형	4
5	입력: cin	4
5.1	여러 값 입력	4
5.2	문자열 입력의 주의점	5
6	산술 연산자	5
6.1	정수 나눗셈	5
6.2	실수 나눗셈과 자동 형변환	5
6.3	복합 대입과 증감	6
7	조건문: if	6
7.1	비교 연산자	6
7.2	if-else	6
7.3	else if	6
7.4	논리 연산자	7
8	반복문: for	7
8.1	누적	7
9	반복문: while	8
9.1	for와 while의 선택	8
9.2	무한 반복	8
9.3	비밀번호 입력 예제	9
10	실습 목록	9
11	핵심 요약	9

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- C++ 프로그램의 기본 구조
- 화면 출력과 사용자 입력
- 변수와 기본 자료형
- 산술 연산자와 자동 형변환
- 조건문 if
- 반복문 for, while

2. C++ 프로그램의 기본 구조

가장 기본적인 C++ 프로그램은 다음과 같다.

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     cout << "Hello, C++!" << endl;
8
9     return 0;
10 }
```

2.1. #include <iostream>

iostream은 입력과 출력 기능을 제공하는 헤더이다. cout, cin, endl을 사용하려면 포함해야 한다.

2.2. using namespace std;

cout의 실제 이름은 std::cout이다. 여기서 std는 C++ 표준 라이브러리의 이름 공간(namespace)이다.

```
1 std::cout << "Hello";
```

using namespace std;를 작성하면 std::를 생략할 수 있다.

2.3. int main()

모든 C++ 프로그램은 main() 함수에서 시작한다. 중괄호 {}는 함수에 포함되는 코드 범위를 나타낸다.

2.4. return 0;

프로그램이 정상적으로 종료되었다는 상태 코드 0을 운영체제에 반환한다.

핵심 포인트

int, if, for, return 등은 C++ 언어 자체의 문법이다. 반면 cout, cin, string, vector 등은 표준 라이브러리에서 제공된다.

3. 출력: cout

출력의 기본 형태는 다음과 같다.

```
1 cout <<
    - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 켜_ - 128) * 64 + ('
```

<<는 오른쪽 데이터를 왼쪽의 출력 스트림으로 전달한다.

```
1 cout << "Age : " << 20 << endl;
```

출력 결과:

Age : 20

3.1. endl과 줄바꿈

```
1 cout << "Hello" << endl;
2 cout << "World" << endl;
```

endl은 줄바꿈을 수행하고 출력 버퍼도 비운다. 단순 줄바꿈에는 \n도 사용할 수 있다.

```
1 cout << "Hello\n";
2 cout << "World\n";
```

3.2. 예제: 학생 정보 출력

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     cout << "===== Student Info =====" << endl;
8     cout << "Name : Alice" << endl;
9     cout << "Major : Physics" << endl;
10    cout << "Grade : 2" << endl;
11    cout << "GPA : 4.00" << endl;
12    cout << "===== " << endl;
13
14    return 0;
15 }
```

4. 변수와 자료형

변수는 값이 저장된 메모리 공간에 붙인 이름이다.

```
1 int age = 20;
```

위 코드는 정수를 저장할 메모리 공간을 만들고, 그 공간에 age라는 이름을 붙인 뒤 20을 저장한다.

4.1. 기본 자료형

자료형	의미	예시
int	정수	20
double	실수	3.14
char	문자 한 개	'A'
bool	참/거짓	true
string	문자열	"Hello"

문자열을 사용할 때는 <string> 헤더를 포함하는 것이 원칙이다.

```

1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main()
7 {
8     string name = "Alice";
9     int age = 20;
10    double gpa = 4.00;
11    char grade = 'A';
12    bool pass = true;
13
14    cout << "Name : " << name << endl;
15    cout << "Age : " << age << endl;
16    cout << "GPA : " << gpa << endl;
17    cout << "Grade : " << grade << endl;
18    cout << "Pass : " << pass << endl;
19
20    return 0;
21 }
```

핵심 포인트

cout << 20;은 값 20을 직접 출력한다. cout << age;는 메모리에서 age에 저장된 값을 읽어서 출력한다.

5. 입력: cin

입력의 기본 형태는 다음과 같다.

```

1 cin >> - 128) * 64 + (' - 128) 꿀 - 128) * 64 + (' - 128) 뽕;

1 int age;
2
3 cout << "Enter age: ";
4 cin >> age;
5
6 cout << "Age : " << age << endl;
```

5.1. 여러 값 입력

```

1 int a;
2 int b;
3
4 cin >> a >> b;

```

다음 입력은 모두 동일하게 처리된다.

10 20

또는

10
20

cin은 공백, 탭, 줄바꿈을 입력 구분자로 사용한다.

5.2. 문자열 입력의 주의점

```

1 string name;
2 cin >> name;

```

사용자가 Hong Gil Dong을 입력하면 name에는 Hong만 저장된다. 공백을 포함한 문자열 전체는 이후에 getline()으로 입력받는다.

6. 산술 연산자

연산자	의미	예시
+	덧셈	a + b
-	뺄셈	a - b
*	곱셈	a * b
/	나눗셈	a / b
%	나머지	a % b

6.1. 정수 나눗셈

```

1 int a = 7;
2 int b = 3;
3
4 cout << a / b << endl;

```

결과는 2이다. 두 피연산자가 모두 정수이므로 소수점 이하가 버려진다.

6.2. 실수 나눗셈과 자동 형변환

```

1 int a = 7;
2 double b = 3;
3
4 cout << a / b << endl;

```

이 경우 a가 자동으로 double로 변환되어 결과는 약 2.33333이 된다.

6.3. 복합 대입과 증감

```

1 x += 3; // x = x + 3
2 x -= 2; // x = x - 2
3 x *= 5; // x = x * 5
4 x /= 2; // x = x / 2
5
6 x++; // x - 128) * 64 + (' - 128) 끝 1
   - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128)
7 x--; // x - 128) * 64 + (' - 128) 끝 1
   - 128) * 64 + (' - 128) - 128) * 64 + (' - 128) 뽕

```

7. 조건문: if

조건문은 조건의 참과 거짓에 따라 서로 다른 코드를 실행한다.

```

1 if ( - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) )
2 {
3     - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 켄 - 128) * 64 + (' - 128) 켄
4     - 128) * 64 + (' - 128) 뽕 - 128) * 64 + (' - 128) 끝;
5 }

```

7.1. 비교 연산자

연산자	의미
==	같다
!=	다르다
>	크다
<	작다
>=	크거나 같다
<=	작거나 같다

핵심 포인트

=는 값을 대입하는 연산자이고, ==는 두 값이 같은지 비교하는 연산자이다.

7.2. if-else

```

1 if (score >= 60)
2 {
3     cout << "Pass" << endl;
4 }
5 else
6 {
7     cout << "Fail" << endl;
8 }

```

7.3. else if

```

1 if (score >= 90)
2 {
3     cout << "A";
4 }

```

```

5 else if (score >= 80)
6 {
7     cout << "B";
8 }
9 else if (score >= 70)
10 {
11     cout << "C";
12 }
13 else
14 {
15     cout << "F";
16 }

```

위에서부터 조건을 검사하며, 처음으로 참이 된 블록 하나만 실행한다.

7.4. 논리 연산자

연산자	의미
&&	AND, 두 조건이 모두 참
	OR, 둘 중 하나 이상 참
!	NOT, 참과 거짓을 반전

8. 반복문: for

반복 횟수를 알고 있을 때 일반적으로 for문을 사용한다.

```

1 for ( - 128) * 64 + ( ' - 128) 뽕 - 128) * 64 + ( ' - 128) - 128) * 64 + ( ' - 128) 뽕;
   - 128) * 64 + ( ' - 128) 뽕 - 128) * 64 + ( ' - 128) - 128) * 64 + ( ' - 128) 뽕;
   - 128) * 64 + ( ' - 128) 뽕 - 128) * 64 + ( ' - 128) - 128) * 64 + ( ' - 128) 뽕)
2 {
3     - 128) * 64 + ( ' - 128) 꿀 - 128) * 64 + ( ' - 128) 꿀 - 128) * 64 + ( ' - 128) 켜
   - 128) * 64 + ( ' - 128) 뽕 - 128) * 64 + ( ' - 128) 꿀;
4 }

```

```

1 for (int i = 1; i <= 5; i++)
2 {
3     cout << i << endl;
4 }

```

실행 순서는 다음과 같다.

1. 초기식 실행
2. 조건식 검사
3. 코드 블록 실행
4. 증감식 실행
5. 조건식으로 돌아감

8.1. 누적

```

1 int sum = 0;
2
3 for (int i = 1; i <= 10; i++)
4 {

```



```

5     sum += i;
6 }
7
8 cout << sum << endl;

```

sum에 이전 결과를 계속 더하는 방식을 누적(accumulation)이라고 한다.

체크 질문

다음 반복문에서 마지막으로 출력되는 값은 4이지만, 조건 검사를 실패할 때의 i 값은 5이다.

```

1 for (int i = 0; i < 5; i++)
2 {
3     cout << i << endl;
4 }

```

9. 반복문: while

while문은 조건이 참인 동안 반복한다.

```

1 while ( - 128) * 64 + ( ' - 128) 뽕 - 128) * 64 + ( ' - 128) )
2 {
3     - 128) * 64 + ( ' - 128) 꿀 - 128) * 64 + ( ' - 128) 꿀 - 128) * 64 + ( ' - 128) 켜
4     - 128) * 64 + ( ' - 128) 뽕 - 128) * 64 + ( ' - 128) 꿀;
5 }

```

```

1 int i = 1;
2
3 while (i <= 5)
4 {
5     cout << i << endl;
6     i++;
7 }

```

9.1. for와 while의 선택

상황	추천 반복문
반복 횟수를 알고 있음	for
반복 횟수를 미리 알 수 없음	while

예를 들어 1부터 100까지 출력할 때는 for가 자연스럽고, 비밀번호를 맞힐 때까지 입력받는 경우에는 while이 자연스럽다.

9.2. 무한 반복

```

1 int i = 1;
2
3 while (i <= 5)
4 {
5     cout << i << endl;
6 }

```

위 코드에서는 i가 변하지 않으므로 조건이 계속 참이다. 따라서 반복문이 끝나지 않는다.

9.3. 비밀번호 입력 예제

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main()
6 {
7     int true_password = 1234;
8     int input_password = 0;
9
10    while (input_password != true_password)
11    {
12        cout << "Enter password: ";
13        cin >> input_password;
14
15        if (input_password == true_password)
16        {
17            cout << "Access Granted" << endl;
18        }
19        else
20        {
21            cout << "Wrong Password" << endl;
22            cout << endl;
23        }
24    }
25
26    return 0;
27 }
```

while은 조건을 먼저 검사하므로, 위 구조에서는 input_password에 초기값을 주어야 한다. 초기화하지 않은 지역 변수에는 예측할 수 없는 값이 들어 있을 수 있다.

10. 실습 목록

실습

Day 1 실습 파일은 다음과 같이 관리한다.

- practice/p01_output.cpp
- practice/p02_variables.cpp
- practice/p03_rectangle_input.cpp
- practice/p04_rectangle_area.cpp
- practice/p05_even_odd.cpp
- practice/p06_sum.cpp
- practice/p07_password.cpp

11. 핵심 요약

1. C++ 프로그램은 main() 함수에서 시작한다.
2. cout은 출력, cin은 입력에 사용한다.

3. 변수는 메모리 공간에 붙인 이름이다.
4. `int / int`의 결과는 정수이며 소수점 이하는 버려진다.
5. 피연산자 중 하나가 `double`이면 자동 형변환이 발생할 수 있다.
6. `if-else if-else`에서는 첫 번째로 참인 블록만 실행된다.
7. 반복 횟수를 알면 `for`, 모르면 `while`이 일반적으로 자연스럽다.
8. 반복문에서는 조건을 변화시키는 코드가 없으면 무한 반복이 발생할 수 있다.